

Coding & Solution

Date	29 September 2026 – 2 October 2026
Team ID	6ab4fddea238dc999eb665e9
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the PocketSmart AI implementation. The solution is a FastAPI + Jinja2 web application that combines budget planning, personalized Gemini AI recommendations, authentication, persistent history and optional multimodal jewelry recommendations.

Instructions:

1. Ensure the FastAPI application, templates, static assets and supporting Python modules are organized and follow the selected technology stack.
2. The solution addresses the core PocketSmart AI requirements: budget planning, personalized recommendations, Home Interior, Party and Jewelry planners, and recommendation history.
3. Include the source files, configuration files, environment template and setup instructions needed to run the application.
4. Attach or link to the PocketSmart AI code repository in the table below.

Solution Summary

Field	Details
Link / URL	PocketSmart-AI GitHub repository (URL to be added)
	Python, HTML5, CSS3, JavaScript
Used	FastAPI, Jinja2, Uvicorn
	User registration/login; Home Interior planner; Party planner; Jewelry planner; budget-aware validation; Gemini AI recommendations; recommendation details; history; optional outfit-image analysis; SQLite persistence.
Complete	Live third-party e-commerce/service APIs are not implemented; external platform links are handled as safe/demo/search links. Production deployment is outside the current student prototype scope.
Instructions	Create a Python environment, install requirements, copy .env.example to .env and add GEMINI_API_KEY when AI access is required. Run FastAPI app with Uvicorn and open the local web application in a browser.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

PocketSmart AI is structured as a modular FastAPI project with separate authentication, configuration, database/models, schemas and Gemini utility modules. The application supports three recommendation domains—Home Interior, Party and Jewelry—and stores relevant user/recommendation history in SQLite. Gemini credentials are supplied through environment variables rather than hard-coded secrets. The implementation includes validation and fallback handling so the application can still run when the Gemini service is unavailable.